ASMO CONTROL TOWER

REST API Integration Guide for UI Developers

Backend	Frontend consumer	API base
Spring Boot + PostgreSQL	React / web UI	http://localhost:8080/api/v1

Purpose. This document gives the UI team the endpoint contract, request parameters, response shapes, interaction patterns, error treatment, ETL status flow, and integration checklist for the ASMO transport dashboard.

Scope. One dashboard covering site, origin and destination, five KPI areas, shipment drill-down, data-quality visibility, and controlled Excel ingestion.

Version 1.0 | 14 October 2026

1. How to use this guide

The UI team should integrate in this order: check service health, verify that data was loaded, retrieve the dashboard summary, retrieve lane and site sections, then enable shipment drill-down. The admin upload endpoint should be kept separate from normal dashboard navigation.

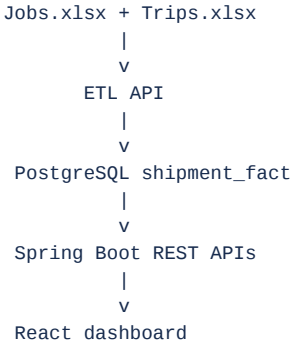
Contents

2. System contract and conventions
3. Common filters and null behavior
4. Dashboard summary API
5. Lane, site and data-quality APIs
6. Shipment list and detail APIs
7. ETL upload and status APIs
8. Errors, security and UI handling
9. React integration pattern
10. Delivery process, testing and presentation
11. Source mapping and assumptions
12. Quick reference and curl examples

Important: Response examples are contract examples. The numbers displayed by the UI must always come from the API response. Do not hard-code the sample KPI values.

2. System contract and conventions

2.1 Architecture



The reporting grain is one shipment record per job. The project requirements describe a trusted shipment view that calculates delivery-window, dwell, transit, SLA and collection-timeliness measures, while keeping excluded records visible with data-quality flags.

2.2 Base URLs

Environment	Base URL
Local API	http://localhost:8080/api/v1
Swagger UI	http://localhost:8080/swagger-ui.html
Health	http://localhost:8080/actuator/health

2.3 Transport and formats

Item	Contract
Protocol	HTTP for local development; use HTTPS outside local development
Payload	JSON, except Excel upload
Upload	multipart/form-data
Dates in query	YYYY-MM-DD
Timestamps in JSON	ISO-8601 local date-time
Pagination	Zero-based page index
Authentication	Not implemented in this POC

2.4 UI principles

- Treat missing KPI percentages as no qualifying data, not zero performance.
- Use the API RAG value for card color when supplied.
- Show stand-in badges when standInApplied is true.
- Display DQ flags in shipment drill-down. Flagged records are not silently hidden.
- Keep admin ETL controls separate from normal operator screens.

3. Common filters and null behavior

3.1 Query parameters

Parameter	Type	Meaning	Example
from	date	Job-created lower bound	2026-09-01
to	date	Job-created upper bound	2026-10-08
collectionSite	string	Exact collection site	AMTPJ
deliverySite	string	Exact delivery site	TESA 2
lane	string	Exact lane label	AMTPJ -> TESA 2
carrier	string	Carrier/transporter code	VTD
priority	string	Effective priority	Normal
distanceBand	string	SLA distance band	<250 km
slaMet	boolean	SLA result filter	true

3.2 Filter rules for React

- Omit empty filters instead of sending empty strings.
- URL-encode lane and site values.
- Use the same filter object for summary, lane, site, DQ and shipment APIs so screen values reconcile.
- Reset page to 0 whenever the user changes a filter.
- Debounce text-based filter changes if requests are sent while typing.

3.3 Null handling

API value	UI behavior
percentage: null	Show em dash or “No qualifying data”; do not render 0%
medianHours: null	Show em dash
slaMet: null	Show “Not evaluated”
inWindow: null	Show “Not evaluated”
empty content array	Show the empty-state component
missing optional timestamp	Show em dash

Example filtered request

```
GET /api/v1/dashboard/summary
    ?from=2026-09-01
    &to=2026-10-08
    &collectionSite=AMTPJ
    &priority=Normal
```

4. Dashboard summary API

GET /api/v1/dashboard/summary

UI use: Populate the five primary KPI areas. Apply the same query parameters used by the rest of the screen.

```
curl.exe "http://localhost:8080/api/v1/dashboard/summary"
```

Response contract


```
{
  "otd": {
    "rawPercentage": 95.0,
    "validatedPercentage": 97.8,
    "delivered": 141,
    "breaches": 7,
    "standInApplied": true,
    "slaVersion": "v1",
    "rag": "AMBER"
  },
  "deliveredWithinWindow": {
    "percentage": 87.9,
    "numerator": 124,
    "denominator": 141,
    "rag": "AMBER"
  },
  "dwell": {
    "collectionMedianHours": 0.35,
    "deliveryMedianHours": 0.81,
    "collectionOverThreshold": 27,
    "deliveryOverThreshold": 10,
    "rag": "GREEN"
  },
  "transit": {
    "medianHours": 3.2,
    "validJobs": 105,
    "excludedJobs": 36
  },
  "collectionTimeliness": {
    "percentage": null,
    "onTime": 0,
    "collected": 0,
    "noPocAlerts": 0,
    "standInApplied": true,
    "rag": "NONE"
  }
}
```


Field interpretation

Field	UI meaning
otd.rawPercentage	SLA compliance across delivered jobs
otd.validatedPercentage	OTD after excluding future-dated events
otd.breaches	Delivered jobs that missed SLA
otd.standInApplied	Show stand-in badge
deliveredWithinWindow	Delivery completed before requested window closed
dwell	Collection and delivery median stop dwell
transit	Valid lane transit median and excluded count
collectionTimeliness	Collection within configured late-POC threshold

Sample values above are acceptance references from the POC specification. The UI must render the live API response.

5. Lane, site and data-quality APIs

5.1 GET /api/v1/lanes/performance

Use for the lane table or chart. The response is an array.

```
[
  {
    "lane": "AMTPJ -> TESA 2",
    "jobs": 41,
    "otdPercentage": 95.1,
    "windowPercentage": 82.9,
    "medianTransitHours": 3.5
  }
]
```

Field	Recommended component
lane	Table label / chart category
jobs	Volume column
otdPercentage	OTD column with RAG styling
windowPercentage	Window compliance column
medianTransitHours	Transit column / trend

5.2 GET /api/v1/sites/performance

```
[
  {
    "site": "AMTPJ",
    "role": "COLLECTION",
    "jobs": 41,
    "otdPercentage": 95.1,
    "windowPercentage": null,
    "medianDwellHours": 0.20
  }
]
```

A site may appear in both roles. Use role to separate collection-site and delivery-site sections.

5.3 GET /api/v1/data-quality/summary

```
{
  "futureDated": 7,
  "reverseSequence": 8,
  "implausibleSpeed": 28,
  "manualCloseOut": 48
}
```

Field	DQ label	UI action
futureDated	DQ-01 Future-dated	Card plus shipment filter/link
reverseSequence	DQ-02 Reverse sequence	Card plus shipment filter/link
implausibleSpeed	DQ-03 Implausible speed	Card plus shipment filter/link
manualCloseOut	DQ-04 Manual close-out	Card plus shipment filter/link

6. Shipment list and detail APIs

6.1 GET /api/v1/shipments

Use for the paginated shipment grid. It accepts common filters plus Spring pagination parameters.

```
GET /api/v1/shipments?page=0&size=25&sort=jobCreatedOn,desc
```

Pagination response


```
{
  "content": [
    {
      "jobId": "20092026020054001",
      "tripId": "trip-...",
      "collectionSite": "AMTPJ",
      "deliverySite": "TESA 2",
      "lane": "AMTPJ -> TESA 2",
      "legKm": 129.743,
      "carrierCode": "ABC",
      "priority": "Normal",
      "distanceBand": "<250 km",
      "createdOn": "2026-09-18T12:47:00",
      "poc": "2026-09-21T18:10:00",
      "pod": "2026-09-21T21:29:00",
      "slaDue": "2026-09-21T12:47:00",
      "slaMet": false,
      "inWindow": true,
      "dataQualityFlags": []
    }
  ],
  "number": 0,
  "size": 25,
  "totalElements": 141,
  "totalPages": 6,
  "first": true,
  "last": false
}
```


Grid recommendations

Column	Render
jobId	Clickable link
lane	Single origin-to-destination column
carrierCode	Text / chip
priority	Chip; show Normal even when stand-in was used
slaMet	Met / Breached / Not evaluated
inWindow	Inside / Late / Not evaluated
dataQualityFlags	Warning chips

6.2 GET /api/v1/shipments/{jobId}

Use when the user opens a shipment drawer or detail page. The response uses the same ShipmentDto shape as a list item.

```
curl.exe "http://localhost:8080/api/v1/shipments/20092026020054001"
```

7. ETL upload and status APIs

7.1 POST /api/v1/admin/etl/upload

This endpoint requires both Excel files in one multipart request. It is intended for an admin or demo-loading screen, not routine operator navigation.

```
curl.exe -X POST "http://localhost:8080/api/v1/admin/etl/upload" ^
  -F "jobs=@C:\data\Jobs_2026-10-08T12_34_07.263Z 1.xlsx" ^
  -F "trips=@C:\data\Trips_2026-10-08T12_48_48.765Z 1.xlsx"
```

Success response

```
{
  "batchId": "7b9a...",
  "status": "SUCCESS",
  "jobsRows": 537,
  "activityRows": 288,
  "stopsRows": 388,
  "allocationRows": 184,
  "shipmentsUpserted": 141,
  "rejectedRows": 0,
  "message": "Loaded both required workbooks"
}
```

UI treatment: Show batch ID, status, source-row counts, shipments upserted, rejected rows and message. Do not claim success unless status is SUCCESS.

7.2 GET /api/v1/admin/etl/status

```
{
  "shipments": 141,
  "batches": 1,
  "lastStatus": "SUCCESS",
  "lastCompletedAt": "2026-10-14T14:30:00",
  "jobsFile": "Jobs_2026-10-08T12_34_07.263Z 1.xlsx",
  "tripsFile": "Trips_2026-10-08T12_48_48.765Z 1.xlsx"
}
```

Suggested admin screen states

State	UI
No previous batch	Upload prompt; dashboard may show no-data state
Upload in progress	Disable submit; show progress indicator
SUCCESS	Show counts and enable dashboard refresh
FAILED	Show API error message and keep prior successful data visible if present
rejectedRows > 0	Show warning with rejected count

8. Errors, security and UI handling

8.1 Error envelope

```
{
  "timestamp": "2026-10-14T09:15:24Z",
  "status": 400,
  "code": "ETL_OR_REQUEST_ERROR",
  "message": "Jobs.xlsx must be .xlsx",
  "path": "/api/v1/admin/etl/upload"
}
```

Status	Typical meaning	UI behavior
200	Successful request	Render data
400	Invalid filter or ETL input	Show returned message near action
404	Shipment ID not found	Show not-found state
500	Unexpected backend error	Show generic retry message; log correlation details if added later

8.2 Security expectations

- The POC has no authentication or SSO. Do not expose the admin ETL endpoint publicly in this state.
- Use HTTPS and an authenticated admin role before deployment outside local development.
- Do not place database credentials in React code.
- The UI should call only REST endpoints; it should never connect directly to PostgreSQL.
- Do not display stack traces or internal SQL errors to users.

8.3 Error handling standard

- Centralize Axios error mapping.
- Use a retry button for read-only GET failures.
- Do not automatically retry Excel uploads because a retry can trigger another batch.
- Preserve the user’s selected filters after a recoverable error.
- Log the endpoint, HTTP status and returned error code in browser diagnostics.

9. React integration pattern

9.1 Suggested project structure

```
src/
  api/
    httpClient.ts
    dashboardApi.ts
    shipmentApi.ts
    etlApi.ts
  components/
    KpiCard.tsx
    DataTable.tsx
    EmptyState.tsx
    ErrorState.tsx
  features/
    dashboard/
    shipments/
    dataQuality/
    adminEtl/
  types/
    api.ts
  pages/
    DashboardPage.tsx
    ShipmentPage.tsx
    AdminLoadPage.tsx
```

9.2 TypeScript contract example

```
export interface DashboardSummary {
  otd: {
    rawPercentage: number | null;
    validatedPercentage: number | null;
    delivered: number;
    breaches: number;
    standInApplied: boolean;
    slaVersion: string;
    rag: 'GREEN' | 'AMBER' | 'RED' | 'NONE';
  };
  deliveredWithinWindow: Ratio;
  dwell: DwellSummary;
  transit: TransitSummary;
  collectionTimeliness: CollectionSummary;
}
```

9.3 Request orchestration

```
const params = removeEmptyFilters(filters);
const [summary, lanes, sites, dq, shipments] = await Promise.all([
  dashboardApi.getSummary(params),
  dashboardApi.getLanes(params),
  dashboardApi.getSites(params),
  dashboardApi.getDataQuality(params),
  shipmentApi.search({ ...params, page: 0, size: 25 })
]);
```

If one section must remain usable when another fails, prefer independent queries or a query library with per-request error states instead of one all-or-nothing Promise.all.

10. Delivery process, testing and presentation

10.1 Numbered delivery process

#	Stage	Expected output
1	Requirement analysis	KPI definitions, source-field mapping, filters, null and DQ rules
2	Architecture	React -> REST -> shipment_fact -> PostgreSQL diagram and environment config
3	React development	KPI cards, filters, lane/site tables, DQ panel, shipment drawer, ETL admin screen
4	Java API development	Versioned endpoints, DTOs, validation, pagination, error envelope, ETL upload/status
5	Integration	Shared filter mapping, CORS verification, URL encoding and no-data behavior
6	Testing	Unit, parser, API, database integration, KPI reconciliation and UI component tests
7	Presentation	Upload both files, show status, KPI screen, filtering, DQ visibility and drill-down

10.2 Quality standards

• Assumptions must be shown: missing ASN uses Job Created On; missing priority uses Normal; Friday and Saturday are non-working days; timestamps are treated as local Saudi time.

• Errors use a consistent JSON envelope. Empty qualifying sets return null KPI values for UI display as an em dash.

• Target at least 80% coverage for service and ETL logic, plus KPI reconciliation checks. This is a project quality target, not a claim about current measured coverage.

• The ETL must retain source filenames, batch ID, rejected-row count and DQ flags for traceability.

• Admin upload requires protection before non-local deployment.

10.3 Presentation scenario

1. Start backend and show health UP.

2. Open ETL Status and show current state.

3. Upload Jobs and Trips workbooks together.

4. Show SUCCESS, source-row counts and shipments upserted.

5. Open dashboard and explain the five KPI areas.

6. Filter one lane or site and show all sections refreshing consistently.

7. Open a breached shipment and explain SLA due, POC, POD and flags.

8. Show DQ cards and explain that bad data is flagged, not deleted.

9. Close with assumptions, reconciliation targets and known POC limitations.

11. Source mapping and assumptions

11.1 Workbook-to-API lineage

Source	Fields used	API impact
Jobs workbook	Job ID, Created On, ASN date, priority, delivery time window	Shipment identity, date filters, stand-ins, SLA and in-window
Trips workbook - Activities	Job ID, trip/stop path, type, actual start/end	Trip link, POC, POD, load start
Trips workbook - Stops	Site name, arrival, departure, planned distance	Origin, destination, lane, dwell, distance band, speed check
Trips workbook - TripAllocation	Last transporter allocation	Carrier and carrier-specific collection/SLA logic

11.2 Business assumptions visible to UI

- standInApplied = true means at least one configured stand-in affected the displayed scope.
- Raw OTD includes delivered shipments; validated OTD excludes DQ-01 future-dated events.
- Transit excludes DQ-01, DQ-02 and DQ-03 records.
- Collection timeliness excludes DQ-01 records.
- DQ flags remain available in drill-down even when the record is excluded from a validated KPI.
- C2 collection timeliness remains subject to business reconciliation in the POC requirements.

11.3 Current POC exclusions

Authentication, SSO, hosting, live MongoDB connectivity, and several broader transport KPIs are outside the current POC specification. Treat any production design for these capabilities as a separate decision.

12. Quick reference

Method	Endpoint	UI use
GET	/actuator/health	Service readiness
GET	/api/v1/dashboard/summary	Five KPI areas
GET	/api/v1/lanes/performance	Lane table/chart
GET	/api/v1/sites/performance	Site table/chart
GET	/api/v1/data-quality/summary	DQ cards
GET	/api/v1/shipments	Paginated grid
GET	/api/v1/shipments/{jobId}	Shipment drawer/detail
POST	/api/v1/admin/etl/upload	Upload both XLSX files
GET	/api/v1/admin/etl/status	Last-load status

Smoke-test commands

```
curl.exe "http://localhost:8080/actuator/health"
curl.exe "http://localhost:8080/api/v1/admin/etl/status"
curl.exe "http://localhost:8080/api/v1/dashboard/summary"
curl.exe "http://localhost:8080/api/v1/lanes/performance"
curl.exe "http://localhost:8080/api/v1/sites/performance"
curl.exe "http://localhost:8080/api/v1/data-quality/summary"
curl.exe "http://localhost:8080/api/v1/shipments?page=0&size=5"
```

Source references

1. ASMO TMS Dashboard POC scope 1.docx - reference IDs turn26search19 / turn26search27 / turn26search58 / turn26search108.

2. KPI Calculations v1.0 1.docx - reference IDs turn26search17 / turn26search22.

3. Power BI & DB - V2 1.docx - reference IDs turn26search18 / turn26search59.

4. Jobs_2026-10-08T12_34_07.263Z 1.xlsx - reference IDs turn26search21 / turn26search25.

5. Trips_2026-10-08T12_48_48.765Z 1.xlsx - reference IDs turn26search20 / turn26search24.

6. ASMO_ControlTower_POC_Plan_v2.pptx - reference IDs turn26search23 / turn26search106.

API endpoint paths, DTO shapes and ETL endpoints document the Spring Boot project generated in the preceding implementation step. Sample KPI acceptance values and business definitions are grounded in the POC sources above.